

CSC 212 Project

Ride-Sharing System

Phase 1

College of Computer and Information Sciences
King Saud University

Fall 2026

1 Introduction

The goal of this programming assignment is to provide hands-on experience in implementing and managing data structures. Students will use the Java programming language to implement and apply the required data structures. This assignment is to be completed in groups of up to three students.

2 Phase 1 description

In this project, you will create a Ride-Sharing System application that stores rider and driver information using a data structure. The application allows users to add, search for, and remove rider and driver records in the system. You will need to implement advanced search functionality that allows users to search for riders and drivers based on different criteria (e.g., name, ID, home city, vehicle type). Additionally, the system should allow scheduling private rides and shared (carpool) rides between riders and drivers.

Classes to be implemented:

1. **Person**: Rider and Driver share a set of common fields and methods (name, phone number, ride history). These are factored out into a shared, abstract **Person** class that both **Rider** and **Driver** extend, so this common behavior only needs to be written once. **Person** is an abstract concept and must not be instantiated directly.
2. **Rider**: This class extends **Person** and represents a single rider in the Ride-Sharing System. In addition to the inherited **Person** fields, it should have fields for the rider's email address and home city. The rider ID is unique and must be provided when the rider is created via the constructor. It must not be modified after creation.
3. **RiderList**: This class will represent the linked list data structure used for storing riders' records. It should have methods for adding, searching, and removing rider records from the list. Rider ID and email are unique, so removing by either one removes at most one rider. Name and home city are not unique: removing by name or by home city must remove every rider that matches, not just the first one found.
4. **Driver**: This class extends **Person** and represents a single driver in the Ride-Sharing System. In addition to the inherited **Person** fields, it should have fields for the driver's

vehicle plate and vehicle type. The driver ID is unique and must be provided when the driver is created via the constructor. It must not be modified after creation.

5. **DriverList**: This class will represent the linked list data structure used for storing drivers' records. It should have methods for adding, searching, and removing driver records from the list. Driver ID and vehicle plate are unique, so removing by either one removes at most one driver. Name and vehicle type are not unique: removing by name or by vehicle type must remove every driver that matches, not just the first one found.
6. **VehicleType**: This enum represents the set of vehicle types a driver's vehicle may be: **SEDAN**, **LUXURY_SEDAN**, **SUV**, and **VAN**. It is used by the **Driver** class instead of a free-text string, so a vehicle's type is validated by the type system rather than by string matching.
7. **DateTime**: This class implements the **IDateTime** interface and represents a single point in time (year, month, day, hour, minute). It is used by **Ride** to represent the pickup time and the drop-off time. Once a **DateTime** is created via its constructor, its fields must not be modified. **DateTime** must implement **Comparable<IDateTime>**, ordering chronologically (year, then month, then day, then hour, then minute), since this ordering is what **schedulePrivateRide** and **scheduleSharedRide** rely on to detect overlapping rides.
8. **Ride**: This abstract class represents a generic ride in the Ride-Sharing System. A ride has common attributes such as a pickup location, pickup date/time, drop-off date/time, drop-off location, and an assigned driver. **Ride** is an abstract concept and must not be instantiated directly. You must implement two concrete ride types: **PrivateRide** and **SharedRide**. A private ride is scheduled with exactly one rider, whereas a shared ride may be scheduled with multiple riders. Once a ride is created, its pickup and drop-off date/time cannot be modified.
9. **RideList**: This class will represent the linked list data structure used for storing the rides. It should have methods for adding, searching, and removing rides from the list.
10. **RideSharingSystem**: This class will represent the Ride-Sharing System application. It coordinates the interaction between **RiderList**, **DriverList**, **RideList** and user-level operations. It manages riders, drivers, and rides and enforces the system rules such as uniqueness of rider IDs, driver IDs, and vehicle plate numbers, existence checks before scheduling, conflict-free scheduling of rides for both the assigned rider(s) and the assigned driver, and cascade removal of rides when a rider or a driver is removed. Every search operation in this class returns the matching data (a single object, or a **LinkedList** of matches) rather than printing it, so each method can be tested directly against its return value.
11. **Main** (command-line menu): This class loads the three CSV files and then presents the user-facing command-line menu specified in Section 4. It is the only class that performs console input/output: it reads user choices, calls the appropriate **RideSharingSystem** method, and formats/prints whatever that method returns.

Your classes should implement the following interfaces:

2.1 IPerson Interface

```
/**  
 * Common fields and behavior shared by every person in the ride-sharing  
 * system (riders and drivers). IRider and IDriver both extend this interface.
```

```

*/
public interface IPerson {

    //return the unique ID of this person.
    int getId();

    //return the full name of this person.
    String getName();

    //Sets the full name of this person.
    void setName(String name);

    //return the phone number of this person.
    String getPhoneNumber();

    /**
     * Sets the phone number of this person.
     * The phone number must be exactly 10 digits (numeric characters only,
     * e.g., "0551234567"). Implementations must validate the format and
     * throw IllegalArgumentException if it does not match.
     */
    void setPhoneNumber(String phoneNumber);

    //return the list of rides associated with this person (as a rider, or as an assigned
    //driver).
    LinkedList<IRide> getRideHistory();

    // Returns a formatted string describing this person.
    @Override
    String toString();
}

```

2.2 IRider Interface

```

/**
 * Represents a single rider in the ride-sharing system.
 * Riders are compared by ID.
 */
public interface IRider extends IPerson, Comparable<IRider> {

    //return the rider's email address.
    String getEmail();

    //Sets the rider's email address.
    void setEmail(String email);

    //return the rider's home city.
    String getHomeCity();

    //Sets the rider's home city.
    void setHomeCity(String homeCity);

    /**
     * Compares this rider with another rider based on rider ID.
     * Returns a negative integer, zero, or a positive integer as this
     * rider's ID is less than, equal to, or greater than the other
     * rider's ID. This ordering must be consistent with equality by ID.
     */
    @Override
    int compareTo(IRider other);
}

```

2.3 IRiderList Interface

```
/**
 * Stores all riders in a structure maintained in sorted order by rider ID.
 */
public interface IRiderList {

    // Inserts a rider into the list in sorted order by ID. If a rider with the same ID
    // already exists, insertion fails.
    boolean add(IRider rider);

    //Searches for a rider by ID.
    IRider findById(int riderId);

    // Searches for all riders with the given full name.
    LinkedList<IRider> findByName(String fullName);

    //Searches for a rider by email address.
    IRider findByEmail(String email);

    //Returns all riders from the specified home city.
    LinkedList<IRider> findByHomeCity(String homeCity);

    //Returns all riders in the linked list.
    LinkedList<IRider> getAll();

    //Removes a rider by ID. Returns true if a rider with that ID was found and removed;
    // false otherwise.
    boolean removeById(int riderId);

    //Removes a rider by email address. Returns true if a rider with that email was found
    // and removed; false otherwise.
    boolean removeByEmail(String email);

    /**
     * Name and home city are not unique, so more than one rider may match.
     * Removes every rider whose full name exactly matches the given name
     * (the same full-name equality used by findByName), not a first-name-only
     * or partial match.
     * Returns the number of riders removed (0 if none matched).
     */
    int removeByName(String fullName);

    /**
     * Removes every rider from the specified home city.
     * Returns the number of riders removed (0 if none matched).
     */
    int removeByHomeCity(String homeCity);

    //return the total number of riders stored.
    int size();
}
```

2.4 VehicleType Enum

```
/**
 * The set of vehicle types a driver's vehicle may be. Used by IDriver in
 * place of a free-text vehicle type string, so vehicle types are validated
 * by the type system instead of by string matching.
 */
public enum VehicleType {
```

```

    SEDAN,
    LUXURY_SEDAN,
    SUV,
    VAN
}

```

2.5 IDriver Interface

```

/**
 * Represents a single driver in the ride-sharing system.
 * Drivers are compared by ID.
 */
public interface IDriver extends IPerson, Comparable<IDriver> {

    /**
     * return the driver's vehicle plate number.
     */
    String getVehiclePlate();

    /**
     * Sets the driver's vehicle plate number.
     * The plate must follow the format of exactly 3 uppercase letters
     * followed by 4 digits (e.g., "ABC1234"). Implementations must
     * validate the format and throw IllegalArgumentException if it does
     * not match. Vehicle plate numbers must be unique across all drivers
     * in the system; enforcing that uniqueness is the responsibility of
     * IDriverList/IRideSharingSystem when a driver is added.
     */
    void setVehiclePlate(String vehiclePlate);

    //return the driver's vehicle type.
    VehicleType getVehicleType();

    //Sets the driver's vehicle type.
    void setVehicleType(VehicleType vehicleType);

    /**
     * Compares this driver with another driver based on driver ID.
     * Returns a negative integer, zero, or a positive integer as this
     * driver's ID is less than, equal to, or greater than the other
     * driver's ID. This ordering must be consistent with equality by ID.
     */
    @Override
    int compareTo(IDriver other);
}

```

2.6 IDriverList Interface

```

/**
 * Stores all drivers in a structure maintained in sorted order by driver ID.
 */
public interface IDriverList {

    // Inserts a driver into the list in sorted order by ID. If a driver with the same ID
    // already exists, insertion fails.
    boolean add(IDriver driver);

    //Searches for a driver by ID.
    IDriver findById(int driverId);

    // Searches for all drivers with the given full name.

```

```

LinkedList<IDriver> findByName(String fullName);

//Searches for a driver by vehicle plate number.
IDriver findByVehiclePlate(String vehiclePlate);

//Returns all drivers with the specified vehicle type.
LinkedList<IDriver> findByVehicleType(VehicleType vehicleType);

//Returns all drivers in the linked list.
LinkedList<IDriver> getAll();

//Removes a driver by ID. Returns true if a driver with that ID was found and removed
; false otherwise.
boolean removeById(int driverId);

//Removes a driver by vehicle plate number. Returns true if a driver with that plate
was found and removed; false otherwise.
boolean removeByVehiclePlate(String vehiclePlate);

/**
 * Name and vehicle type are not unique, so more than one driver may match.
 * Removes every driver whose full name exactly matches the given name
 * (the same full-name equality used by findByName), not a first-name-only
 * or partial match.
 * Returns the number of drivers removed (0 if none matched).
 */
int removeByName(String fullName);

/**
 * Removes every driver with the specified vehicle type.
 * Returns the number of drivers removed (0 if none matched).
 */
int removeByVehicleType(VehicleType vehicleType);

//return the total number of drivers stored.
int size();
}

```

2.7 IDateTime Interface

```

/**
 * Minimal date/time abstraction used by private rides and shared rides.
 * Implementations must define chronological ordering via compareTo.
 */
public interface IDateTime extends Comparable<IDateTime> {

    //return year (e.g., 2026).
    int getYear();

    //return month in [1..12].
    int getMonth();

    //return day in [1..31].
    int getDay();

    //return hour in [0..23].
    int getHour();

    //return minute in [0..59].
    int getMinute();
}

```

```

//Formats this date/time for display (e.g., "MM/DD/YYYY HH:MM").
String format();

/**
 * Compares this date/time with another chronologically: by year, then
 * month, then day, then hour, then minute. Returns a negative integer,
 * zero, or a positive integer as this date/time is earlier than, equal
 * to, or later than the other date/time. This ordering is what
 * schedulePrivateRide and scheduleSharedRide must use to detect whether
 * two time ranges overlap.
 */
@Override
int compareTo(IDateTime other);
}

```

2.8 IRide Interface

```

/**
 * Represents a generic ride in the ride-sharing system. Set the pickup and drop-off
 * Date/Time values only in the constructor.
 */
public interface IRide extends Comparable<IRide> {

    // Returns the unique internal ride ID.
    int getRideId();

    // Returns the pickup location of the ride.
    String getPickupLocation();

    // Sets the pickup location of the ride.
    void setPickupLocation(String pickupLocation);

    // Returns the pickup date/time of the ride.
    IDateTime getPickupTime();

    // Returns the drop-off date/time of the ride.
    IDateTime getDropoffTime();

    // Returns the drop-off location of the ride.
    String getDropoffLocation();

    // Sets the drop-off location of the ride.
    void setDropoffLocation(String dropoffLocation);

    // Returns the driver assigned to this ride.
    IDriver getDriver();

    // Sets the driver assigned to this ride.
    void setDriver(IDriver driver);

    // Checks whether a rider participates in this ride.
    boolean hasRider(int riderId);

    // Returns a formatted string describing the ride.
    @Override
    String toString();

    // Compares rides alphabetically by pickup location.
    @Override
    int compareTo(IRide other);
}

```

```

/**
 * Represents a private ride involving exactly one rider.
 */
public interface IPrivateRide extends IRide {

    // Returns the rider assigned to this private ride.
    IRider getRider();

    // Sets the rider assigned to this private ride.
    void setRider(IRider rider);
}

/**
 * Represents a shared ride (carpool) involving multiple riders.
 */
public interface ISharedRide extends IRide {

    // Returns the list of shared ride participants.
    LinkedList<IRider> getParticipants();

    // Adds a rider to the shared ride.
    boolean addParticipant(IRider rider);

    // Removes a rider from the shared ride by ID.
    boolean removeParticipantById(int riderId);

    // Returns true if the shared ride has no participants.
    boolean isEmpty();
}

```

2.9 IRideList Interface

```

/**
 * Stores all rides in a structure maintained in alphabetical order by pickup location.
 */
public interface IRideList {

    //Adds a ride and maintains alphabetical ordering by pickup location.
    boolean addRide(IRide ride);

    //Removes a ride by its unique ride ID.
    boolean removeRideById(int rideId);

    //Returns all rides alphabetically ordered by pickup location.
    LinkedList<IRide> getAllAlphabetically();

    //Returns all rides whose pickup location matches the given location.
    LinkedList<IRide> findByPickupLocation(String pickupLocation);

    //Returns all rides that involve a rider with the given full name.
    LinkedList<IRide> findByRiderName(String riderFullName);

    //return number of rides stored.
    int size();
}

```

2.10 IRideSharingSystem Interface

```

/**
 * The interface of the Ride-Sharing System.

```

```

*/
public interface IRideSharingSystem {
    // Loads riders from a CSV file.
    // Returns true if loading succeeds; false otherwise.
    boolean loadRidersFromCSV(String ridersFilePath);

    // Loads drivers from a CSV file.
    // Returns true if loading succeeds; false otherwise.
    boolean loadDriversFromCSV(String driversFilePath);

    // Loads rides from a CSV file. All referenced riders and the assigned driver must
    // already exist. It must enforce conflict rules
    // Returns true if loading succeeds; false otherwise.
    boolean loadRidesFromCSV(String ridesFilePath);

    //Adds a rider to the system after enforcing uniqueness by ID. Returns true if added;
    // false if duplicate ID or invalid
    boolean addRider(IRider rider);

    /**
     * Adds a driver to the system after enforcing uniqueness by ID and by
     * vehicle plate number (no two drivers may share the same plate).
     * Returns true if added; false if duplicate ID, duplicate plate, or invalid
     */
    boolean addDriver(IDriver driver);

    //Searches for a rider by ID.
    IRider searchRiderById(int riderId);

    //Searches for a rider by email.
    IRider searchRiderByEmail(String email);

    //Searches for all riders whose full name exactly matches the given name.
    LinkedList<IRider> searchRidersByName(String fullName);

    //Searches for all riders from the specified home city.
    LinkedList<IRider> searchRidersByHomeCity(String homeCity);

    //Returns all riders stored in the system.
    LinkedList<IRider> getAllRiders();

    //Searches for a driver by ID.
    IDriver searchDriverById(int driverId);

    //Searches for a driver by vehicle plate.
    IDriver searchDriverByVehiclePlate(String vehiclePlate);

    //Searches for all drivers with the specified vehicle type.
    LinkedList<IDriver> searchDriversByVehicleType(VehicleType vehicleType);

    //Returns all drivers stored in the system.
    LinkedList<IDriver> getAllDrivers();

    /**
     * Removes a rider and performs cascade deletion of associated rides:
     * - All private rides involving the rider are deleted.
     * - The rider is removed from any shared rides.
     * - Shared rides with no remaining participants are deleted.
     * It returns true if the rider was found and removed; false otherwise
     */
}

```

```

boolean removeRider(int riderId);

/**
 * Removes a driver and performs cascade deletion of their assigned rides:
 * - All private rides assigned to the driver are deleted.
 * - All shared rides assigned to the driver are deleted.
 * It returns true if the driver was found and removed; false otherwise
 */
boolean removeDriver(int driverId);

/**
 * Schedules a private ride for one rider with one driver.
 * Requirements:
 * - The rider must exist.
 * - The driver must exist.
 * - The ride must not conflict with the rider's current schedule.
 * - The ride must not conflict with the driver's current schedule.
 * It returns true if scheduled; false otherwise
 */
boolean schedulePrivateRide(String pickupLocation, IDateTime pickupTime, IDateTime
    dropoffTime, String dropoffLocation, int riderId, int driverId);

/**
 * Schedules a shared ride for multiple riders with one driver.
 * Requirements:
 * - All riders must exist.
 * - The driver must exist.
 * - The ride must not conflict with any listed rider's schedule.
 * - The ride must not conflict with the driver's current schedule.
 * It returns true if scheduled; false otherwise
 */
boolean scheduleSharedRide(String pickupLocation, IDateTime pickupTime, IDateTime
    dropoffTime, String dropoffLocation, int[] riderIds, int driverId);

//Searches for all rides whose pickup location matches the given location (may return
    multiple if pickup locations repeat).
LinkedList<IRide> searchRidesByPickupLocation(String pickupLocation);

//Searches for all rides that involve a rider with the given name.
LinkedList<IRide> searchRidesByRiderName(String riderName);

//Returns all riders registered on a shared ride identified by its pickup location.
LinkedList<IRider> getSharedRideParticipants(String pickupLocation);

//Returns all rides (private and shared) alphabetically ordered by pickup location.
LinkedList<IRide> getAllRidesAlphabetically();
}

```

3 Project Requirements

- The Rider, Driver, Ride, and DateTime classes should implement the Comparable interface.
- The Ride-Sharing System should provide a user-friendly interface through a command-line menu, implemented in a separate Main class, exactly as specified in Section 4. This class is the only place in the project that reads user input or prints output: it calls the RideSharingSystem's methods and formats/prints whatever they return.
- Each rider in the Ride-Sharing System should be unique. You can ensure this by checking

if a rider with the same rider ID already exists in the list before adding a new rider. The same uniqueness rule applies to drivers and driver IDs.

- Each driver's vehicle plate number must also be unique across the system: no two drivers may be added with the same plate. A vehicle plate must follow the format of exactly 3 uppercase letters followed by 4 digits (e.g., ABC1234).
- Phone numbers (for both riders and drivers, via `Person.setPhoneNumber`) must be exactly 10 digits, numeric characters only (e.g., 0551234567). An invalid phone number or vehicle plate should cause the setter to throw `IllegalArgumentException`.
- Use the provided CSV files (`riders_100.csv`, `drivers_30.csv`, and `rides_40.csv`) to upload the information related to the riders, drivers, and rides to the ride-sharing system. See Section 5 for the expected file layout.
- When a rider is removed, all private rides scheduled with that rider are removed. If the rider is registered on any shared ride, the rider must be removed from that shared ride. If a shared ride has no riders remaining, it should be removed.
- When a driver is removed, all rides (private or shared) assigned to that driver are removed.
- Before scheduling a private ride, the rider and the driver must exist in their respective lists. Before scheduling a shared ride, all listed riders and the assigned driver must exist.
- There should be no conflict in ride scheduling. A new ride must not be scheduled for a rider if it conflicts with an existing ride for that rider. A new ride must not be scheduled for a driver if it conflicts with an existing ride already assigned to that driver. Two rides conflict if their time intervals overlap.
- Listing alphabetically all riders, all drivers, or all rides must be implemented with a worst-case time complexity of $O(n)$.
- `compareTo` in `Rider` and in `Driver` must order strictly by the unique ID (rider ID / driver ID): negative if this object's ID is smaller, zero if equal, positive if larger. This ordering must be consistent with equality by ID.
- `compareTo` in the class implementing `IDateTime` must order chronologically: by year, then month, then day, then hour, then minute. Use it to detect the overlap required by the no-conflict rule above: two time ranges $[pickup_1, dropoff_1)$ and $[pickup_2, dropoff_2)$ overlap exactly when $pickup_1 < dropoff_2$ and $pickup_2 < dropoff_1$, where $<$ is evaluated with `compareTo`.
- `RiderList.removeByName`, `RiderList.removeByHomeCity`, `DriverList.removeByName`, and `DriverList.removeByVehicleType` remove by a field that is *not* unique. Unlike `removeById`, which stops after at most one match, these methods must traverse the full list and remove every record that matches, returning the count of records removed. Stopping after the first match is an incorrect implementation and will fail on test data containing duplicates. Matching on name uses the same exact full-name-match rule as `findByName` — not the first name alone and not a partial/substring match.
- You must not modify the provided interfaces or the method signatures specified in them

Notes

Your code should be written with proper indentation, clear comments, and appropriate naming for variables, methods, and classes. All of these points will be graded.

`LinkedList` refers to the custom implementation developed in class. The use of Java's built-in `java.util.LinkedList` (or any other collection from `java.util`) is not permitted. All data structures used in this assignment must be implemented by you. You're not allowed to use any structure from the collection library.

4 Command-Line Menu

The menu covers the display operations plus three add operations (adding a rider, a driver, or a ride).

Before showing the menu, `Main` must automatically load `riders_100.csv`, `drivers_30.csv`, and `rides_40.csv` (calling `loadRidersFromCSV`, `loadDriversFromCSV`, and `loadRidesFromCSV`) so the menu always has real data to display; this startup load is not itself a menu option.

The `Main` class must then present the exact numbered list of options below, in this order, every time the program runs. This is a requirement, not a suggestion: every team's menu must look and behave the same way, so that the demo lines up option-by-option with what is graded. Each option corresponds to exactly one `RideSharingSystem` method; it is your job to identify the matching method and call it (prompting the user for whatever input it needs).

For the search/list options (all options below except Add a rider, Add a driver, and Add a ride), the corresponding method returns a `LinkedList<T>`: print every element's `toString()`, one per line, or a clear *no results* message if the list is empty. For the three add options, prompt for whatever fields the corresponding method's parameters require, construct the object(s) it needs (a `Rider`, a `Driver`, or the appropriate ride type), call the matching method, and print a clear message stating whether the operation succeeded or failed. For Add a ride specifically, first ask the user whether the ride is private or shared, then prompt for the fields that ride type's scheduling method requires (a single rider ID for a private ride; more than one rider ID for a shared ride) and call `schedulePrivateRide` or `scheduleSharedRide` accordingly.

The exact wording of these messages is your choice; what is graded is that every option below is present, in this order, and produces the correct outcome of the corresponding `RideSharingSystem` call. Option 0 exits the program and is not part of the numbered sequence.

1. List all riders.
2. Search for riders by home city.
3. Search for riders by name.
4. Add a rider.
5. List all drivers.
6. Search for drivers by vehicle type.
7. Add a driver.
8. List all rides alphabetically by pickup location.
9. Search for rides by pickup location.

10. Search for rides by rider name.
 11. List the participants of a shared ride by pickup location.
 12. Add a ride (ask the user whether it is private or shared, then prompt accordingly).
0. Exit the program.

5 CSV File Formats

riders_100.csv — one rider per line:

```
riderId,name,email,phoneNumber,homeCity
```

drivers_30.csv — one driver per line. **vehicleType** must be one of SEDAN, LUXURY_SEDAN, SUV, VAN:

```
driverId,name,phoneNumber,vehiclePlate,vehicleType
```

rides_40.csv — one ride per line. **rideType** is either PRIVATE or SHARED. **riderIds** lists one rider ID for a private ride, or multiple rider IDs separated by a semicolon for a shared ride. Date/time fields use the format MM/DD/YYYY HH:MM:

```
rideType,pickupLocation,pickupDateTime,dropoffDateTime,  
dropoffLocation,driverId,riderIds
```

6 The report

Prepare a written report (in PDF format). Your report should include a cover sheet with the names and IDs of your team members, as well as the division of work among them.

Provide the time complexity (worst-case analysis) of all methods discussed above using Big-O notation. Your answer should show the steps taken to calculate the growth rate function and the final Big-O result.

7 Submission guidelines

1. The submission **deadline** is: **October 17th**.
2. Students are expected to work in teams, with each team containing no more than three members. All team members must share the same instructor.
3. Submit a written report (in PDF format), in addition to submitting your source code (compressed in ZIP format).
4. After submission, you will present a demo that demonstrates your working code to your instructor during a scheduled session.
5. Any member of the group who fails to attend the demonstration without a proper excuse (consult the university and college regulations) shall receive the mark 0 in the project.
6. All submissions should be made through the LMS.

7. Every member of the group must participate in all parts of the project. Members of the same group may receive different marks according to their participation in the project.
8. The submitted work must be your own. Any case of unethical conduct will result in a zero grade in the project.